

Università degli Studi di Bari Aldo Moro
Laurea Magistrale in Data Science

PROGETTO DEEP LEARNING

**Link Prediction in Apulia Open Data
via Graph Autoencoding con
Relational Convolutions e ConvE**

Studente: Anisa Bakiu

Matricola: 836273

Gennaio 2026

Indice

1	Motivazione teorica	1
1.1	Introduzione	1
1.2	I grafi	2
1.2.1	La matrice di adiacenza	2
1.3	Tipi e strutture dei grafi	3
1.4	Task di apprendimento nei grafi	4
1.5	I knowledge graphs	4
1.5.1	I componenti principali di un knowledge graph	5
1.6	Knowledge Graph Embedding	6
1.6.1	Le caratteristiche dei KGE	6
1.6.2	Modelli di decomposizione tensoriale	7
1.6.3	Modelli di reti neurali convoluzionali	8
1.6.4	Metriche di valutazione	9
1.7	Deep Learning Geometrico	10
1.7.1	Graph Neural Network	11
1.7.2	Meccanismo di message passing	11
1.7.3	Apprendimento delle rappresentazioni degli archi	13
1.7.4	Graph Convolutional Network	14
1.7.5	Relational Graph Convolutional Network	16
2	Descrizione del dataset	21
2.1	Introduzione al dataset Apulia Travel Open Data	21
2.2	Suddivisione dei dati	22
3	Approccio metodologico	25
3.1	Graph Autoencoder con R-GCN + DistMult	25
3.2	Baseline ConvE	26
3.3	Utilizzo di PyKEEN	27
3.4	Ottimizzazione degli iperparametri (HPO)	28
3.4.1	Tuning del R-GCN	28
3.4.2	Tuning del ConvE	28

4	Valutazione sperimentale	29
4.1	Risultati dell'HPO per entrambi i modelli	29
4.2	Risultati finali sui test set	30
5	Discussione	32
5.1	Interpretazione dei risultati	32
5.2	Considerazioni sulla stabilità dei modelli	33
	Riferimenti	34

1 — Motivazione teorica

Prima di presentare le analisi effettuate, ritengo opportuno fornire un'introduzione alle motivazioni teoriche che guidano il lavoro e spiegare il funzionamento del modello utilizzato. In questo modo, fornisco un contesto chiaro prima di entrare negli aspetti applicativi.

1.1 Introduzione

I grafi sono ovunque intorno a noi, gli oggetti del mondo reale sono spesso definiti in termini di connessioni con altri oggetti. Un insieme di oggetti e le connessioni tra di essi sono naturalmente espressi come un grafo. I ricercatori hanno sviluppato reti neurali che operano su dati in forma di grafo per oltre un decennio. Ad. es. le reti neurali a grafo (GNN) sono un'implementazione dell'apprendimento profondo geometrico [10].

Generalmente, esistono tre tipi di task di previsione sui grafi: a livello di grafo, a livello di nodo ed a livello di arco.

Per un task a livello di grafo, prevediamo una proprietà per l'intero grafo. Per i task a livello di nodo, prevediamo una proprietà per ogni nodo, ed a livello di arco, si prevede la proprietà o la presenza di archi in un grafo.

Questi tre livelli di previsione descritti sopra, possono essere risolti con una singola classe di modelli, la GNN.

Lo sviluppo delle GNN si è ispirato ad algoritmi di apprendimento profondo come le reti neurali convoluzionali (CNN) e le reti neurali ricorrenti (RNN), ma presentano diverse differenze fondamentali.

Le CNN sono progettate per dati che presentano una struttura a griglia, come i pixel di un'immagine. Le RNN sono progettate per strutture di sequenze in cui

ogni elemento può essere connesso al massimo ad altri due elementi, come una stringa di parole in un testo. Nei dati strutturati a grafo, tuttavia, qualsiasi elemento può avere una o più connessioni e potrebbe non esserci coerenza tra il numero di connessioni per un elemento specifico [7].

1.2 I grafi

Un insieme di oggetti e le connessioni tra di essi possono essere rappresentati come un grafo.

Un grafo rappresenta le relazioni, chiamate *archi*, tra un insieme di entità, chiamate *nodi*.

Matematicamente, un grafo G è definito come una tupla di un insieme di nodi e vertici V e di un insieme di archi e collegamenti.

$$G = (V, E)$$

Ogni arco è una coppia di due vertici e rappresenta una connessione tra di essi. Per descrivere meglio ciascun nodo, arco o l'intero grafo, è possibile associare informazioni a ognuno di questi elementi. La matrice di adiacenza di un grafo orientato rappresenta tutti i vertici e indica la presenza e la direzione degli archi che li collegano.

1.2.1 La matrice di adiacenza

Si pone la domanda: *Possiamo semplicemente usare la matrice di adiacenza come input per una rete neurale profonda?*

Un'idea ragionevole per definire una rete neurale profonda su grafi sarebbe quella di utilizzare semplicemente la matrice di adiacenza come input per una rete neurale profonda. Ad esempio, per generare un embedding di un intero grafo, potremmo semplicemente appiattire la matrice di adiacenza e inviare il risultato a un perceptron multistrato.

Il problema con tali approcci è che dipendono dall'ordinamento arbitrario dei

nodi che abbiamo utilizzato nella matrice di adiacenza. Più formalmente, il problema è che un tale modello non è equivariante per permutazione rispetto all'ordinamento dei nodi, il che significa che l'output cambia quando permutiamo le righe/colonne della matrice.

Un requisito fondamentale per la progettazione di reti neurali su grafi è che siano equivarianti per permutazione, e questa è una sfida dell'apprendimento su grafi rispetto a oggetti come immagini o sequenze [6].

1.3 Tipi e strutture dei grafi

Per scoprire la struttura del grafo nell'applicazione, di solito ci sono due scenari:

- strutturali
- non strutturali

Negli scenari strutturali, la struttura del grafo è esplicita nelle applicazioni, come nei knowledge graphs. Invece, negli scenari non strutturali, i grafi sono impliciti, quindi dobbiamo prima costruirli. Una volta ottenuto il grafo, il processo di progettazione successivo cerca di trovare un modello GNN ottimale su questo specifico grafo.

I grafi sono generalmente classificati come:

- Grafi orientati/non orientati: Gli archi nei grafi orientati sono tutti orientati da un nodo all'altro, il che fornisce maggiori informazioni rispetto ai grafi non orientati.
- Grafi omogenei/eterogenei. Nodi e archi hanno tipi diversi nei grafi eterogenei, e svolgono un ruolo importante nei grafi eterogenei e dovrebbero essere ulteriormente considerati.

Si noti che queste categorie sono ortogonali, il che significa che questi tipi possono essere combinati, ad esempio è possibile gestire un grafo eterogeneo ed orientato.

1.4 Task di apprendimento nei grafi

Le tecniche di Deep Learning e non solo, consentono diversi tipi di analisi nei grafi, ognuno dei quali fornisce informazioni su diversi elementi dei dati [7].

A livello di grafo

Questo tipo di riconoscimento del modello può essere inquadrato come una forma di classificazione dei grafici, poiché classifica l'intero grafico.

A livello di nodo

Le attività a livello di nodo riguardano la previsione dell'identità o del ruolo di ciascun nodo all'interno di un grafo. Si concentrano sui nodi, e includono la classificazione dei nodi, la regressione, il clustering, ecc. La classificazione dei nodi cerca di categorizzarli in diverse classi, dove i nodi simili dovrebbero essere nello stesso gruppo. La regressione invece, prevede un valore continuo per ciascun nodo. Il clustering mira a suddividere i nodi in diversi gruppi disgiunti.

A livello di archi

Questi task riguardano la classificazione degli archi, link prediction, che richiedono al modello di classificare i tipi delle relazioni o di prevedere se esiste un arco tra due nodi dati [13].

1.5 I knowledge graphs

Un knowledge graph è una rappresentazione strutturata della conoscenza composta da entità, relazioni e relativi attributi.

Viene tipicamente visualizzato come un grafo, in cui i nodi rappresentano entità (ad esempio, persone, luoghi, concetti) e gli archi rappresentano le relazioni tra queste entità. Le relazioni possono essere etichettate con proprietà o attributi per fornire maggiore contesto e informazioni.

I knowledge graph vengono utilizzati per archiviare e organizzare le informazioni in modo da renderle facilmente accessibili e comprensibili sia per gli esseri umani che per le macchine. Possono essere impiegati in varie applicazioni come la ricerca semantica, i sistemi di raccomandazione, l'elaborazione del linguaggio

naturale e l'intelligenza artificiale per migliorare la qualità e la pertinenza dei risultati [3].

1.5.1 I componenti principali di un knowledge graph

I componenti principali di un knowledge graph sono:

1. **Entità:** le entità sono gli elementi fondamentali di un knowledge graph. Rappresentano oggetti, concetti o idee astratte del mondo reale come persone, luoghi, organizzazioni, eventi o persino concetti intangibili come emozioni o idee. Le entità sono tipicamente rappresentate come nodi nel grafo.
2. **Relazioni:** le relazioni, note anche come archi o collegamenti, collegano le entità nel knowledge graph. Rappresentano le associazioni, le connessioni o le interazioni tra le entità, fornendo contesto e struttura alle informazioni. Le relazioni possono essere di vario tipo, come *è un/una*, *ha*, *creato/a da*, *situato/a in* e molte altre.
3. **Attributi:** gli attributi, noti anche come proprietà o caratteristiche, forniscono informazioni aggiuntive su entità e relazioni. Descrivono caratteristiche, qualità o proprietà di entità e relazioni, aggiungendo ulteriore contesto e dettagli al grafo. Ad esempio, un'entità che rappresenta una persona potrebbe avere attributi come nome, età o professione, mentre un attributo di relazione potrebbe indicare la forza o la sicurezza di una connessione.
4. **Ontologie:** le ontologie sono rappresentazioni formali della conoscenza che definiscono la struttura, le relazioni e i vincoli di un dominio specifico. Forniscono un vocabolario condiviso e un insieme di regole per organizzare e categorizzare entità, relazioni e attributi in un grafo della conoscenza. Le ontologie contribuiscono a garantire la coerenza, migliorare l'interoperabilità e consentire il ragionamento e l'inferenza sul grafo.
5. **Etichette e identificatori:** etichette e identificatori vengono utilizzati per identificare e descrivere in modo univoco entità e relazioni in un grafo della conoscenza. Forniscono nomi leggibili per entità e relazioni, nonché iden-

tificatori univoci che possono essere utilizzati per indicizzare, interrogare e collegare dati tra diverse fonti.

1.6 Knowledge Graph Embedding

Gli embedding svolgono un ruolo cruciale nel deep learning sui knowledge graph. Gli embedding sono rappresentazioni vettoriali continue e a bassa dimensionalità di entità (nodi) e relazioni (archi) in un knowledge graph.

Nel representation learning, l'incorporamento del knowledge graph embeddings (KGE), chiamato anche apprendimento della rappresentazione della conoscenza (KRL) o apprendimento multi-relazione, consiste nell'apprendere una rappresentazione a bassa dimensione delle entità e delle relazioni di un grafico della conoscenza preservandone il significato semantico.

Sono progettati per catturare la struttura, la semantica e i pattern complessi del grafo in una forma densa e compatta. Lo scopo principale degli embedding è trasformare i dati discreti e sparsi del grafo in uno spazio vettoriale continuo e denso, che può essere più facilmente elaborato e analizzato dagli algoritmi di machine learning, in particolare dai modelli di deep learning.

Sfruttando la loro rappresentazione embedded, i Knowledge graphs (KG) possono essere utilizzati per varie applicazioni come la link prediction, la classificazione delle triple, node classification, clustering e relation extraction [3].

1.6.1 Le caratteristiche dei KGE

Un embedding di un knowledge graph è caratterizzato da quattro aspetti:

- Spazio di rappresentazione: lo spazio a bassa dimensionalità in cui sono rappresentate entità e relazioni.
- Funzione di punteggio: una misura della bontà di una rappresentazione di triple embedded.
- Modelli di codifica: la modalità in cui le rappresentazioni embedded delle entità e delle relazioni interagiscono tra loro.

- Informazioni aggiuntive: qualsiasi informazione aggiuntiva proveniente dal knowledge graph che possa arricchire la rappresentazione embedded.

Tutti gli algoritmi per la creazione di un embedding del knowledge graph seguono lo stesso approccio.

Innanzitutto, i vettori di embedding vengono inizializzati a valori casuali. Quindi, vengono ottimizzati iterativamente utilizzando un training set di triple.

In ogni iterazione, un batch di triple di dimensione b viene campionato dal training set e una tripla da esso viene campionata e corrotta, ovvero una tripla che non rappresenta un fatto vero nel knowledge graph.

La corruzione (*corruption*) di una tripla comporta la sostituzione del oggetto o del soggetto (o entrambi) della tripla con un'altra entità che rende il fatto falso.

La tripla originale e la tripla corrotta vengono aggiunte al training batch e quindi gli embedding vengono aggiornati, ottimizzando una funzione di punteggio. L'iterazione si interrompe quando viene raggiunta una condizione di stop.

1.6.2 Modelli di decomposizione tensoriale

Rossi et al. [8] propongono una tassonomia per i KGE e identificano tre principali famiglie di modelli: **modelli di decomposizione tensoriale, modelli geometrici e modelli di Deep Learning**. La decomposizione tensoriale è una famiglia di modelli di embedding che utilizzano una matrice multidimensionale per rappresentare un grafo di conoscenza.

In particolare, utilizzano un tensore di terzo ordine (3D), che viene poi fattorizzato in vettori a bassa dimensionalità che costituiscono gli embeddings.

Un tensore di terzo ordine è adatto a rappresentare un grafo di conoscenza perché registra solo l'esistenza o l'assenza di una relazione tra entità, ed è quindi semplice, e non è necessario conoscere a priori la struttura della rete [9].

Modelli bilineari

Questo sottoinsieme di modelli di decomposizione tensoriale si basa su una formulazione lineare per rappresentare il modo in cui due entità sono connesse attraverso una relazione. In tali modelli, ogni relazione viene rappresentata medi-

ante una matrice bidimensionale (o un vettore strutturato), che funge da trasformazione lineare applicata alle rappresentazioni delle entità coinvolte.

Un limite strutturale di questi approcci è che, durante la fase di apprendimento degli embedding, essi considerano ogni tripla (s, r, o) in modo indipendente. Ciò significa che utilizzano esclusivamente il singolo fatto osservato per aggiornare le rappresentazioni, ignorando eventuali altre connessioni che la stessa entità o relazione può avere nel grafo. Di conseguenza, questi modelli tendono a catturare bene regolarità locali ma possono perdere dipendenze strutturali globali.

DistMult

Nel modello DistMult, ogni relazione è rappresentata da una matrice diagonale. Questo implica che la trasformazione applicata alle entità è limitata a operazioni elemento per elemento, senza capacità di modellare interazioni più complesse tra le dimensioni dell'embedding. A causa di questa struttura diagonale, la funzione di punteggio risulta intrinsecamente simmetrica. Ad esempio, il punteggio della tripla (s, r, o) è identico a quello di (o, r, s) . Per questo motivo, DistMult non è in grado di distinguere relazioni asimmetriche.

1.6.3 Modelli di reti neurali convoluzionali

Questa famiglia di modelli, invece di utilizzare fully connected layers, impiega uno o più livelli convoluzionali che convolgono i dati di input applicando un filtro a bassa dimensionalità in grado di incorporare strutture complesse con pochi parametri mediante l'apprendimento di caratteristiche non lineari.

ConvE

ConvE è un modello di embedding che rappresenta un buon compromesso tra espressività dei modelli di deep learning e costo computazionale, infatti è dimostrato che utilizza 8 volte meno parametri rispetto a DistMult.

ConvE utilizza vettori di embedding monodimensionali (1D), di dimensione d (ogni vettore ha d componenti) per rappresentare sia le entità che le relazioni all'interno del grafo. La procedura con cui il modello calcola la funzione di punteggio di una tripla (h, r, t) è articolata in più passaggi.

Per prima cosa, gli embedding dell'entità testa h e della relazione r vengono concatenati e ristrutturati in un'unica matrice bidimensionale $[h; r]$.

Questa matrice funge da input per un livello convoluzionale 2D, che consente al modello di catturare interazioni locali e pattern complessi tra le dimensioni dei due embedding.

L'output dello strato convoluzionale viene poi appiattito e passato attraverso un layer denso (lineare), parametrizzato da una matrice di pesi W , che applica una trasformazione lineare utile a proiettare la rappresentazione in uno spazio compatibile con l'embedding delle entità.

Infine, la rappresentazione ottenuta viene confrontata con l'embedding dell'entità di coda t tramite un prodotto interno, producendo così il punteggio della tripla, che indica quanto essa sia plausibile secondo il modello. [4].

1.6.4 Metriche di valutazione

Questi indici sono spesso utilizzati per misurare la qualità di embedding di un modello. La semplicità degli indici li rende molto adatti per valutare le prestazioni di un algoritmo anche su larga scala.

Dato Q come l'insieme di tutte le previsioni classificate di un modello, è possibile definire tre diversi indici di performance: **Hits@K**, **MR** e **MRR** [2].

Hits@K

Hits@K o in breve, H@K, è un indice di prestazione che misura la probabilità di trovare la previsione corretta nelle prime K previsioni del modello.

Solitamente, viene utilizzato $k = 10$. Hits@K riflette l'accuratezza di un modello di embedding nel prevedere correttamente la relazione tra due triple date.

$$\text{Hits@K} = \frac{|\{q \in Q : q < k\}|}{|Q|} \in [0, 1]$$

Valori più grandi indicano prestazioni predittive migliori.

Mean Rank

Il rango medio è la posizione media degli elementi previsti dal modello tra tutti gli elementi possibili.

$$MR = \frac{1}{|Q|} \sum_{q \in Q} q$$

Più piccolo è il valore, migliore è il modello.

Mean Reciprocal Rank

Il Mean Reciprocal Rank misura il numero di triple previste correttamente. Se la prima tripla prevista è corretta, viene aggiunto 1, se la seconda è corretta vengono sommati 1/2 e così via [2]. Il rango reciproco medio viene generalmente utilizzato per quantificare l'effetto degli algoritmi di ricerca.

$$MR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{q} \in [0, 1]$$

Quanto più grande è l'indice, tanto migliore è il modello.

1.7 Deep Learning Geometrico

Questo concetto è stato introdotto nel 2016 e comprende tecniche emergenti che cercano di generalizzare modelli neurali profondi (strutturati) a domini non euclidei come grafi e grid.

Il deep learning geometrico estende gli approcci tradizionali di deep learning per gestire dati che non rientrano in strutture fisse o a griglia come immagini o testo. Il deep learning geometrico si concentra sulla comprensione delle relazioni tra i punti dati, indipendentemente dalla loro disposizione spaziale. Questo lo rende particolarmente efficace per l'analisi di strutture complesse, dove le connessioni tra gli elementi sono più importanti della loro posizione specifica [1].

1.7.1 Graph Neural Network

Una GNN è una trasformazione ottimizzabile su tutti gli attributi (nodi, archi, contesto globale) che preserva le simmetrie del grafo (invarianze per permutazione).

Le GNN adottano un'architettura *graph-in, graph-out*, il che significa che questi tipi di modelli accettano un grafo come input, con informazioni nei suoi nodi, archi e contesto globale, e trasformano progressivamente questi embedding, senza modificare la connettività del grafo di input [10].

In genere, l'addestramento di una rete neurale avviene aggiornando i parametri di rete con gradienti calcolati su un sottoinsieme randomizzato dei dati di addestramento. Per addestrare le reti neurali a grafo, si creano sottografi che preservano le proprietà essenziali del grafo principale.

Le GNN utilizzano un meccanismo di scambio di messaggi per aggregare le informazioni dai nodi adiacenti, consentendo loro di catturare le complesse relazioni nei grafi.

Pertanto, una sfida nella modellazione di dati strutturati a grafo è catturare le interazioni tra i nodi. Lo scambio di messaggi neurali affronta questa sfida fornendo un framework per la modellazione di dipendenze e interazioni nei dati del grafo. I nodi scambiano informazioni con i nodi adiacenti e aggregano tali informazioni per aggiornare le proprie rappresentazioni [7].

1.7.2 Meccanismo di message passing

La caratteristica più distintiva di una GNN è che utilizza una forma di scambio di messaggi neurali in cui i messaggi vettoriali vengono scambiati tra i nodi del grafo e aggiornati utilizzando reti neurali [6].

In altre parole, se un particolare nodo X è connesso a 4 nodi adiacenti, vogliamo in qualche modo che le informazioni provenienti dai 4 nodi connessi si riflettano nell'embedding del nodo X . Questi 4 nodi potrebbero essere connessi a diversi altri nodi e le informazioni provenienti da questi nodi dovrebbero fluire verso di essi e riflettersi nei loro embedding.

Inoltre, ora che i 4 nodi connessi del nodo X hanno aggiornato gli embedding,

dobbiamo considerare l'aggiornamento dell'embedding del nodo X per riflettere queste nuove informazioni.

Questo flusso di informazioni è chiamato *passaggio di messaggi* e avviene tramite operazioni di aggregazione come somma, media, ecc.

Ciò che accade è che per ogni nodo, gli embedding di tutti i nodi connessi vengono sommati (ipotizzando un'operazione di somma) e questa somma viene fatta riflettere nell'embedding del nodo stesso. In pratica, aggiorniamo gli embedding del nodo sorgente per riflettere (anche) gli embedding dei nodi vicini utilizzando un'operazione di aggregazione. Lo facciamo per tutti i nodi del grafo. Quindi ricogliamo nuovamente l'embedding di ciascun nodo (poiché gli embedding dei vicini sarebbero stati aggiornati).

Ripetiamo l'operazione più volte fino a raggiungere la stabilità. Questo è il cuore di una GNN.

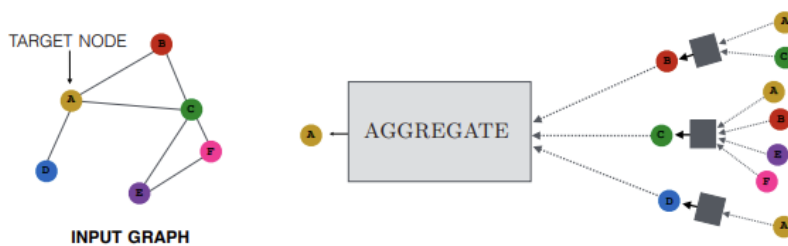


Figure 1.1: Meccanismo di message passing

Il passaggio di messaggi funziona in tre fasi:

1. Per ogni nodo del grafo, si raccolgono tutti gli embedding (o messaggi) dei nodi adiacenti.
2. Si aggregano tutti i messaggi tramite una funzione di aggregazione (come la somma).
3. Tutti i messaggi raggruppati vengono passati tramite una funzione di aggiornamento, solitamente una rete neurale appresa.

Ogni livello (strato) equivale a un'iterazione di Message Passing tradizionale. Possiamo aggiungere tanti livelli nella GNN quanti sono il numero di iterazioni che desideriamo. In pratica, questi livelli e la matematica al suo interno svolgono

il compito di Message Passing. Con ogni strato, i nostri embedding si rafforzano. Esistono lievi variazioni nel passaggio di messaggi a seconda dell'architettura. Una semplice tecnica chiamata **self-loop** consiste nel disegnare un arco immaginario per far sì che un nodo si connetta a se stesso, includendo di fatto le caratteristiche del nodo stesso nella fase di aggregazione.

Per essere precisa, dopo la prima iterazione ($k = 1$), ogni embedding di nodo contiene informazioni dal suo vicinato a 1 salto, ovvero ogni embedding di nodo contiene informazioni sulle caratteristiche dei suoi vicini immediati del grafo, che possono essere raggiunti da un cammino di lunghezza 1 nel grafo; dopo la seconda iterazione ($k = 2$) ogni embedding di nodo contiene informazioni dal suo vicinato a 2 salti; e in generale, dopo k iterazioni ogni embedding di nodo contiene informazioni sul suo vicinato a k salti.

Dopo t iterazioni di passaggio di messaggi GNN, gli embedding di ciascun nodo codificano quindi anche informazioni su tutte le caratteristiche nel loro vicinato $t - hop$. Questo comportamento di aggregazione locale delle caratteristiche delle GNN è analogo al comportamento dei kernel convoluzionali nelle reti neurali convoluzionali (CNN). Tuttavia, mentre le CNN aggregano le informazioni sulle caratteristiche da patch spazialmente definite in un'immagine, le GNN aggregano le informazioni sulla base di vicini locali del grafo [6].

1.7.3 Apprendimento delle rappresentazioni degli archi

Possiamo incorporare anche le informazioni dagli archi adiacenti nello stesso modo utilizzato in precedenza per le informazioni sui nodi adiacenti, raggruppando prima le informazioni sugli archi, trasformandole con una funzione di aggiornamento e memorizzandole.

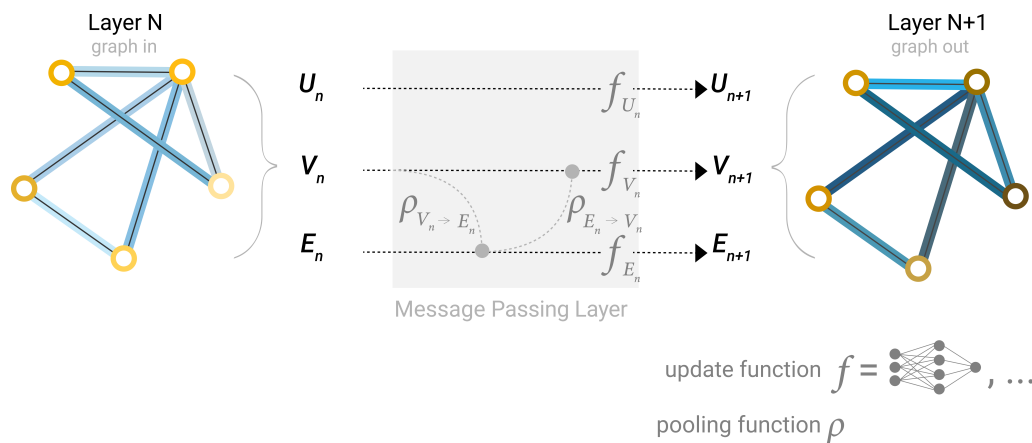


Figure 1.2: Apprendimento delle rappresentazioni degli archi

1.7.4 Graph Convolutional Network

Uno dei modelli di rete neurale a grafo di base più diffusi, la rete convoluzionale a grafo (GCN), impiega l'aggregazione normalizzata di Kipf e l'approccio di aggiornamento a ciclo automatico.

La differenza fondamentale tra le architetture GNN risiede solitamente nel modo in cui gestiscono lo scambio di messaggi. GCN utilizza operazioni di convoluzione per propagare le informazioni tra i nodi di un grafo.

L'input è un grafo in cui ogni nodo riceve un embedding iniziale basato sui suoi attributi. L'output è (solitamente) lo stesso grafo ma con embedding aggiornati. La differenza fondamentale sta nel modo in cui viene eseguita l'aggregazione [13].

La GCN più semplice ha tre livelli:

1. Livello convoluzionale: questo livello esegue la convoluzione su ciascun nodo per apprenderne le connessioni.
2. Livello di attivazione non lineare: questo livello applica una funzione di attivazione come ReLU all'output della convoluzione.
3. Livello lineare di output: questo livello finale somma gli output per generare una previsione finale.

Innanzitutto, viene eseguita una convoluzione utilizzando ciascun nodo nel grafo del layer di convoluzione. Questo layer convoluzionale utilizza le informazioni

sulle caratteristiche dei vicini di ciascun nodo e le aggrega, aggiornando i pesi associati a ciascun nodo. Quindi, una funzione di attivazione non lineare, come ReLU, viene applicata all'output del livello di convoluzione.

Infine, un layer lineare di output viene utilizzato per prevedere a quale classe è più probabile che un grafo appartenga.

Nel contesto della link prediction, la GCN non produce direttamente una previsione, ma apprende rappresentazioni latenti dei nodi. La probabilità di esistenza di un arco tra due nodi viene successivamente stimata da un decoder che opera sugli embedding appresi. Il modello GCN definisce quindi la funzione di passaggio dei messaggi come

$$\mathbf{h}_u^{(k+1)} = \sigma \left(\sum_{v \in \mathcal{N}(u) \cup \{u\}} \frac{1}{\sqrt{|\mathcal{N}(u)| |\mathcal{N}(v)|}} \mathbf{W}^{(k)} \mathbf{h}_v^{(k)} \right)$$

Questo approccio è stato delineato per la prima volta da Kipf e Welling [2016] e si è dimostrato una delle architetture GNN di base più diffuse ed efficaci. Ogni layer della GCN aggiorna la rappresentazione di un nodo aggregando le informazioni dai nodi vicini e da sé stesso, normalizzando il contributo in base al grado dei nodi [6].

Problema del over-smoothing

Nel modello GNN di base, la rappresentazione nascosta aggiornata $\mathbf{h}_u^{(t)}$ di un nodo u viene calcolata a ogni passo del message passing tramite una combinazione lineare del messaggio aggregato $\mathbf{m}_{\mathcal{N}(u)}$ proveniente dai vicini del nodo u e della precedente rappresentazione nascosta del nodo $\mathbf{h}_u^{(t-1)}$. Sebbene semplice ed efficace in molti contesti, questo approccio di aggiornamento di base presenta inconvenienti.

In particolare, combinando linearmente $\mathbf{m}_{\mathcal{N}(u)}$ con $\mathbf{h}_u^{(t-1)}$, le informazioni provenienti dalle varie iterazioni del message passing diventano *entangled* dopo ogni ciclo di propagazione. Quando si utilizza questa semplice combinazione lineare, la GNN non ha un modo diretto per distinguere le informazioni locali specifiche di un determinato ciclo di message passing dalle informazioni non locali ricevute nelle successive iterazioni.

Questo *entanglement* lineare può portare, tra le altre cose, a un problema noto come *over-smoothing*. Dopo un elevato numero di iterazioni di message passing, le rappresentazioni di tutti i nodi del grafo tendono a diventare molto simili tra loro.

Inoltre, il semplice aggiornamento utilizzato nell'approccio GNN di base può portare a instabilità numeriche (ad esempio gradienti esplosivi) dovute alla ripetuta applicazione di moltiplicazioni di matrici a ogni livello. Questo problema è analogo alle instabilità osservate nei modelli di reti neurali ricorrenti (RNN) di base. Il problema dell'*over-smoothing* nelle GNN può essere formalizzato definendo l'influenza di ciascuna caratteristica di input del nodo $\mathbf{h}_u^{(0)} = \mathbf{x}_u$ sull'embedding finale di tutti gli altri nodi del grafo, ossia $\mathbf{h}_v^{(K)}$ per ogni $v \in \mathcal{V}$. In particolare, per qualsiasi coppia di nodi u e v , l'influenza del nodo u sul nodo v può essere quantificata analizzando la norma della matrice Jacobiana corrispondente:

$$\mathcal{I}_K(u, v) = \mathbf{1}^\top \left(\frac{\partial \mathbf{h}_v^{(K)}}{\partial \mathbf{h}_u^{(0)}} \right) \mathbf{1},$$

dove $\mathbf{1}$ è un vettore composto interamente da uno. Il valore $\mathcal{I}_K(u, v)$ utilizza la somma degli elementi della matrice Jacobiana come misura di quanto l'embedding iniziale del nodo u influenzi l'incorporamento finale del nodo v nella GNN.

Pertanto, quando si utilizzano modelli GNN semplici, in particolare quelli basati su aggiornamenti con self-loop, l'aumento della profondità del modello può effettivamente compromettere le prestazioni. All'aumentare del numero di layer, le informazioni sulle strutture di vicinato locale vengono progressivamente perse e gli embedding appresi diventano eccessivamente lisce, tendendo verso una distribuzione quasi uniforme [6].

1.7.5 Relational Graph Convolutional Network

Multi-relational GNNs

Finora in questa analisi delle reti neurali di tipo GNN e dello scambio di messaggi neurali implicitamente ho dato per scontato che si tratti di grafi semplici. Tuttavia, esistono numerose applicazioni in cui i grafi in questione sono multirelazionali o comunque eterogenei, ad esempio, i grafi della conoscenza (KG).

I metodi tradizionali per le knowledge graphs, si basano in genere su funzionalità create manualmente, ragionamento logico o tecniche di machine learning superficiali. Al contrario, i metodi di deep learning apprendono automaticamente rappresentazioni complesse ed espressive dai dati, senza la necessità di un'ingegneria manuale.

Il primo approccio proposto per affrontare il problema dei grafi eterogenei è noto come **Relational Graph Convolutional Network (R-GCN)**.

Relational Graph Convolution Network (RGCN) è un'implementazione estesa della Graph Convolution Network (GCN). Nella GCN, la rete di grafi non tiene conto delle relazioni tra entità in una base di conoscenza, il che significa che gli archi tra i nodi non vengono considerati. Tuttavia, negli scenari reali, le relazioni (che sono gli archi in un grafo) hanno i propri embedding che devono essere appresi per fornire risultati allo stato dell'arte.

In questo approccio, la funzione di aggregazione viene estesa per gestire più tipi di relazione, introducendo una matrice di trasformazione distinta per ciascun tipo di arco [12, 11].

In particolare, il messaggio aggregato per il nodo u è definito come:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{\tau \in \mathcal{R}} \sum_{v \in \mathcal{N}_{\tau}(u)} \frac{1}{f_n(u, v)}, \mathbf{W}_{\tau} \mathbf{h}_v$$

dove:

- $\mathcal{R}$ indica l'insieme dei tipi di relazione;
- $\mathcal{N}_{\tau}(u)$ rappresenta l'insieme dei vicini del nodo u connessi tramite una relazione di tipo τ ;
- $\mathbf{W}_{\tau}$ è la matrice di pesi associata alla relazione τ ;
- $f_n(u, v)$ è una funzione di normalizzazione, che può dipendere sia dal nodo u sia dal nodo v .

Nel complesso, l'aggregazione multirelazionale nelle R-GCN è concettualmente simile a quella delle GNN con normalizzazione, con la differenza fondamentale che **le informazioni vengono aggregate separatamente per ciascun tipo di relazione**, consentendo al modello di catturare pattern strutturali specifici dei grafi

eterogenei.

In una GCN standard, la rappresentazione nascosta per ciascun nodo i al layer $(l + 1)$ viene calcolata come:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in N_i} \frac{1}{c_i} \mathbf{W}^{(l)} \mathbf{h}_j^{(l)} \right)$$

dove c_i è una costante di normalizzazione.

In R-GCN, diversi tipi di archi utilizzano pesi diversi, e tutti gli archi dello stesso tipo di relazione condividono lo stesso peso di proiezione $\mathbf{W}_r^{(l)}$. La rappresentazione nascosta delle entità nello strato $(l + 1)$ in R-GCN può quindi essere formulata come:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\mathbf{W}_0^{(l)} \mathbf{h}_i^{(l)} + \sum_{r \in R} \sum_{j \in N_i^r} \frac{1}{c_{i,r}} \mathbf{W}_r^{(l)} \mathbf{h}_j^{(l)} \right)$$

dove N_i^r indica l'insieme dei vicini del nodo i con relazione $r \in R$ e $c_{i,r}$ è una costante di normalizzazione.

Problema del overfitting

Un problema dell'applicazione diretta di questa equazione è la rapida crescita del numero di parametri, soprattutto in dati altamente multi-relazionali.

Per ridurre la complessità del modello e prevenire l'overfitting, l'articolo originale propone l'uso di una **decomposizione di base**. Quando il numero di relazioni è molto elevato, il numero di parametri nel modello esplode. Pertanto, vengono introdotti due tipi di regolarizzazione per ridurre il numero di parametri nella modellazione delle relazioni:

1. decomposizione in base
2. decomposizione diagonale a blocchi

Con la decomposizione in base, ciascuna matrice $\mathbf{W}_r^{(l)}$ è definita come:

$$\mathbf{W}_r^{(l)} = \sum_{b=1}^B a_{rb}^{(l)} \mathbf{V}_b^{(l)}$$

Qui, ogni $\mathbf{W}_r$ è una combinazione lineare di matrici di base $\mathbf{V}_b \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ con coefficienti $a_{rb}^{(l)}$.

Nella decomposizione diagonale a blocchi, R-GCN definisce ciascuna $\mathbf{W}_r$ tramite una somma diretta di matrici a bassa dimensionalità, richiedendo un numero di parametri maggiore rispetto alla decomposizione di base [6].

2 — Descrizione del dataset

2.1 Introduzione al dataset Apulia Travel Open Data

Il dataset utilizzato in questo lavoro "Apulia Travel Open Data", è dedicato alla descrizione di punti di interesse (turistici) nella regione Puglia. I motivi per cui ho scelto questo dataset sono principalmente due. Innanzitutto si tratta di dati reali della Regione di Puglia, che rappresentano una sfida maggiore rispetto ai dataset giocattoli. Questo mi permette di sperimentare problemi reali di link prediction. Inoltre, le dimensioni del dataset non sono eccessive, evitando di richiedere risorse computazionali troppo elevate che non sarebbero disponibili. I dati sono rappresentati sotto forma di **Knowledge Graph RDF**, in cui le informazioni sono modellate tramite ontologie semanticamente strutturate e istanze concrete (*ABox*).

Il grafo include una varietà di punti di interesse (*Point of Interest, POI*), quali musei, chiese, castelli, aree naturali e attrazioni culturali, arricchiti da informazioni contestuali come localizzazione geografica, contatti, condizioni di accesso e meta-dati temporali.

Il dataset è organizzato secondo un approccio **ontology-driven**. In particolare:

1. Le classi rappresentano concetti semantici (ad es. Museum, Castle, Beach, PointOfInterest, Address, City).
2. Le istanze rappresentano entità reali (specifici musei, castelli, indirizzi, ecc.).
3. Le relazioni modellano i legami semantici tra le entità (ad es. hasAddress, hasPOICategory, situatedWithin).

Dal punto di vista della Knowledge Representation, il dataset costituisce una ABox, mentre le ontologie di riferimento forniscono la struttura concettuale (TBox). Il grafo include classi come: categorie turistiche, entità spaziali e amministrative, informazioni di servizio, entità temporali. Le classi sono state codificate tramite un mapping numerico per consentire l'utilizzo nei modelli di apprendimento.

Per quanto riguarda le relazioni, il dataset comprende diversi tipi che descrivono: associazioni spaziali (hasAddress, situatedWithin), categorizzazione dei POI ad es. (hasPOICategory), informazioni di accesso (hasAccessCondition, hasAdmissionType), metadati temporali (atTime, fallsInside), collegamenti a contenuti multimediali e social (hasImage, hasReview, hasRating).

Questa ricchezza relazionale rende il grafo altamente eterogeneo, motivando l'uso di modelli relazionali come R-GCN. Il preprocessing del dataset segue la pipeline proposta nel repository ufficiale [5], includendo l'estrazione delle triple RDF, codifica numerica di nodi e relazioni, costruzione del grafo diretto e la suddivisione delle triple in training, validation e test set.

Il grafo risultante viene utilizzato come input per il task di Link Prediction.

2.2 Suddivisione dei dati

La Tabella 2.1 riporta le principali statistiche del dataset dopo la suddivisione in training, validation e test set. Il grafo presenta un numero elevato di entità, con oltre 29 000 nodi nel training set, e più di 65 000 triple.

Table 2.1: Statistiche del dataset

	Train	Valid	Test
Entità	29 767	4 966	8 069
Relazioni	25	20	19
Triple (Edges)	65 401	3 847	7 695
Grado medio	2.20	0.77	0.95

Il numero di relazioni varia tra gli split, riflettendo la natura eterogenea del grafo e il fatto che non tutte le tipologie relazionali sono osservabili in ogni partizione.

Questa caratteristica rende il task di Link Prediction particolarmente difficile, in quanto il modello deve generalizzare anche su relazioni poco rappresentate.

Un aspetto rilevante è la bassa connettività del grafo, come evidenziato dal grado medio. In particolare, il valore di 2.20 nel training set e i valori inferiori a 1 nei set di validazione e test indicano una struttura fortemente sparsa, tipica dei knowledge graph reali. Tale sparsità motiva l'adozione di modelli relazionali capaci di sfruttare l'informazione strutturale e semantica del grafo.

La Figura 2.1 illustra un estratto delle triple RDF presenti nel training set, per fornire una rappresentazione della complessità semantica degli URI.

Figure 2.1: Esempio di triple RDF estratte dal training set

```
1 https://w3id.org/italia/onto/CLV/Address/31266_Mura ,
   _abitazioni , _sepulture_e_Mappa_di_Soleto_address https
   ://w3id.org/italia/onto/CLV/hasAddressComponent https
   ://w3id.org/italia/onto/CLV/StreetToponym/31266_Mura ,
   _abitazioni , _sepulture_e_Mappa_di_Soleto_street_toponym
2 https://w3id.org/italia/onto/SM/Review/32085
   _Porticciolo_di_San_Nicola_tradition_spirituality_review
   https://w3id.org/italia/onto/SM/hasRating https://w3id
   .org/italia/onto/SM/Rating/1
3 https://w3id.org/italia/onto/CLV/OpeningHours/32025
   _Torre_Marchiello_opening_hours_morning https://w3id.
   org/italia/onto/AccessCondition/isAccessConditionOf
   https://w3id.org/italia/onto/POI/PointOfInterest/32025
   _Torre_Marchiello
4 https://w3id.org/italia/onto/POI/PointOfInterest/32109
   _Santa_Caterina_di_Nardo https://w3id.org/italia/onto/
   SM/hasReview https://w3id.org/italia/onto/SM/Review
   /32109_Santa_Caterina_di_Nardo_events_review
5 https://w3id.org/italia/onto/SM/Review/31723
   _Piazza_del_Popolo_theatre_dance_review https://w3id.
   org/italia/onto/SM/hasRating https://w3id.org/italia/
   onto/SM/Rating/1
6 https://w3id.org/italia/onto/SM/WebSite/https%3A%2F%2Fwww.
   comune.brindisi.it%2Fhh%2Findex.php https://w3id.org/
   italia/onto/SM/isWebSiteOf https://w3id.org/italia/onto
   /SM/ContactPoint/30797
   _Chiesa_di_Santa_Maria_del_Romitorio_(
   o_Romitorio_dei_Vescovi)_contact_point
```

3 — Approccio metodologico

L'obiettivo è la previsione di link tra entità del grafo, formulata come problema di Link Prediction su Knowledge Graph. Dato un tripletta incompleta $(h, r, ?)$ o $(?, r, t)$, il modello deve stimare l'entità più probabile.

3.1 Graph Autoencoder con R-GCN + DistMult

Per affrontare questo problema, adottiamo l'approccio di tipo graph autoencoder, composto da due componenti principali: un encoder e un decoder [11].

L'encoder ha il compito di apprendere una rappresentazione vettoriale continua per ciascuna entità del grafo, sfruttando la struttura relazionale e le informazioni topologiche disponibili.

Il decoder, invece, utilizza tali rappresentazioni per ricostruire gli archi del grafo, assegnando un punteggio alle triple (soggetto, relazione, oggetto).

A differenza di molti approcci classici alla link prediction, nei quali le rappresentazioni delle entità sono apprese direttamente come parametri liberi del modello, in questo lavoro le embedding delle entità sono calcolate tramite un encoder neurale.

In particolare, utilizzo una Relational Graph Convolutional Network (R-GCN) come encoder, che permette di integrare in modo esplicito i diversi tipi di relazione presenti nel grafo. Le rappresentazioni delle entità vengono ottenute attraverso un processo di message passing multilayer rendendo l'encoder sensibile sia alla struttura del grafo sia alla semantica delle relazioni.

Una volta ottenute le rappresentazioni vettoriali delle entità tramite l'encoder R-GCN, la valutazione delle triple candidate è affidata a una funzione di scoring, che svolge il ruolo di decoder. Ho deciso di adottare DistMult come funzione di

scoring (come nel paper originale di R-GCN [11]).

L'addestramento del modello avviene tramite negative sampling, dove per ogni tripla osservata vengono generate ω triple negative corrompendo casualmente il soggetto o l'oggetto, con probabilità 50% ciascuno. Il modello viene ottimizzato per assegnare punteggi più elevati alle triple osservate rispetto a quelle negative, utilizzando una funzione di perdita basata sul ranking, MarginRankingLoss.

3.2 Baseline ConvE

Come baseline ho scelto ConvE, un modello che utilizza convoluzioni 2D sugli embedding per predire i link mancanti nei Knowledge Graphs. ConvE è l'architettura convoluzionale multilayer più semplice per la previsione dei link. Come introdotta anche prima, è definita da un singolo layer di convoluzione, un layer di proiezione sulla dimensione di embedding e un layer di prodotto interno [4]. L'uso di convoluzioni 2D anziché 1D aumenta l'espressività del modello, grazie a ulteriori punti di interazione tra gli embedding. Ad esempio, concatenando due righe di embedding 1D a e b , ciascuna di dimensione $n = 3$:

$$([a \ a \ a]; [b \ b \ b]) = [a \ a \ a \ b \ b \ b]$$

Una convoluzione 1D *padded* con filtro $k = 3$ può modellare le interazioni intorno al punto di concatenazione (numero di interazioni proporzionale a k).

Concatenando invece due righe di embedding 2D di dimensione $m \times n$, con $m = 2$ e $n = 3$:

$$\begin{bmatrix} a & a & a \\ a & a & a \end{bmatrix}; \begin{bmatrix} b & b & b \\ b & b & b \end{bmatrix} = \begin{bmatrix} a & a & a \\ a & a & a \\ b & b & b \\ b & b & b \end{bmatrix}$$

Una convoluzione 2D *padded* con filtro 3×3 modella le interazioni lungo tutta la linea di concatenazione (numero di interazioni proporzionale a n e k). Questo principio si estende anche a pattern alternati, ad esempio:

$$\begin{bmatrix} a & a & a \\ b & b & b \\ a & a & a \\ b & b & b \end{bmatrix}$$

In questo caso, la convoluzione 2D cattura ancora più interazioni tra a e b (numero di interazioni proporzionale a m , n e k). Pertanto, le convoluzioni 2D estraggono più interazioni tra feature rispetto alle convoluzioni 1D [4].

L'obiettivo finale è che le triple osservate nel dataset (positive) abbiano score alto, e le triple negative (generate tramite negative sampling) abbiano score basso. Pertanto, si tratta di fare una classificazione binaria per ogni tripla: vera (1) o falsa (0).

Quindi la loss utilizzata sarà `BCEWithLogitsLoss` che combina la sigmoid con la cross-entropy. Questa loss è più naturale per `ConvE` perché stiamo prevedendo la probabilità di ogni tripla individualmente, non confrontando uno score positivo con uno negativo come nel margin ranking loss.

In `R-GCN` con `DistMult` invece, il modello produce embeddings e si usa il margin ranking loss per spingere lo score delle triple positive sopra quello delle negative.

3.3 Utilizzo di PyKEEN

Per questo lavoro, ho utilizzato `PyKEEN 1.11.1` con `Python 3.10`. Prima di addestrare i modelli di *link prediction*, le triple del dataset sono state convertite in un formato compatibile con `PyKEEN` tramite la classe `TriplesFactory`. Ogni split del dataset (*training*, *validation* e *test*) è stato trattato separatamente, generando per ciascuno le triple costituite da soggetto, relazione e oggetto.

Per il *training*, la funzione di `convert` genera automaticamente le mappature tra entità, relazioni e i rispettivi ID numerici. Per gli split di *validation* e *test* vengono invece utilizzate le stesse mappature ottenute dal training, così da garantire coerenza tra i diversi set e permettere ai modelli di confrontare correttamente le previsioni con i dati osservati. Questo approccio è stato applicato sia per l'addestramento del modello **R-GCN** sia per **ConvE**, assicurando che le triple

fossero coerenti tra i diversi modelli e in questo modo, ogni modello riceve le stesse informazioni di base sulle entità e le relazioni.

3.4 Ottimizzazione degli iperparametri (HPO)

Per entrambe le architetture considerate, **ConvE** e **R-GCN**, tutti gli iperparametri principali sono stati determinati tramite un processo di *Hyperparameter Optimization (HPO)* implementato in *PyKEEN*. Questo approccio consente di identificare automaticamente le configurazioni di iperparametri che massimizzano la performance dei modelli sul set di validazione. L'ottimizzazione è stata condotta massimizzando la metrica *Mean Reciprocal Rank (MRR)*.

3.4.1 Tuning del R-GCN

Per R-GCN, il tuning ha riguardato:

- la dimensione degli embedding, `embedding_dim`,
- il tasso di apprendimento dell'ottimizzatore Adam `1r`

3.4.2 Tuning del ConvE

Per ConvE, gli iperparametri ottimizzati sono stati:

- la dimensione degli *embedding* (`embedding_dim`),
- il dropout sull'input (`input_dropout`),
- il dropout sulle feature map (`feature_map_dropout`),
- il dropout sull'output (`output_dropout`),
- il tasso di apprendimento dell'ottimizzatore Adam (`1r`)

Per entrambi i modelli, il numero di epoche e la dimensione del batch sono stati fissati rispettivamente a 30 e 1024, mentre il numero di configurazioni candidate esplorate (*trials*) è stato pari a 5.

4 — Valutazione sperimentale

4.1 Risultati dell'HPO per entrambi i modelli

Da questo esperimento, i migliori iperparametri per il modello **RGCN** sono stati:

- una dimensione degli embedding di 128, con 2 layer e senza bias
- la funzione di attivazione utilizzata è stata LeakyReLU
- il dropout sugli archi è risultato nullo mentre quello sui self-loop è stato impostato a 0.4
- per i pesi degli archi è stata scelta l'opzione `inverse_out_degree`,
- la decomposizione dei pesi è stata di tipo block.
- la loss margin ottimale è stata 0.717
- il learning rate 0.0056
- il numero di negativi per positivo 4

I migliori iperparametri per il modello **ConvE** sono stati:

- una dimensione degli embedding di 128 e 32 canali di output
- il dropout sugli input è stato impostato a 0.153, , quello sugli output a 0.120
- il feature map dropout a 0.202
- il learning rate ottimale è risultato 0.0051
- il numero di negativi per positivo è stato 5.

4.2 Risultati finali sui test set

Di seguito vengono riportate le principali metriche di performance dei modelli e i tempi di addestramento e valutazione.

Le tabelle 4.1 e 4.2 mostrano rispettivamente le metriche dove valori maggiori o minori indicano migliori performance.¹

Table 4.1: Metriche Hits@K

Model	Hits@1	Hits@3	Hits@5	Hits@10
R-GCN	0.090	0.148	0.174	0.220
ConvE	0.125	0.170	0.190	0.218
Delta Δ	-0.035	-0.022	-0.016	+0.002

Table 4.2: Metriche di Rank

Model	Median Rank	AMR	GMR	IHR
R-GCN	133	2727.368	155.451	0.137
ConvE	563	7329.143	360.108	0.160
Delta Δ	-430	-4601.775	-204.657	-0.023

La Tabella 4.3 riassume i tempi di training ed evaluation.

Table 4.3: Tempi di addestramento e valutazione dei modelli

Modello	Training (s)	Evaluation (s)	Totale (s)
R-GCN	979.5	51.66	1031.16
ConvE	31	5810.22	5841.22
Delta Δ	+948.5	-5758.56	-4810.06

I grafici 4.1 e 4.2 offrono una rappresentazione visiva delle metriche Hits@K.

¹In *PyKEEN*, l'Inverse Harmonic Mean Rank (IHR) è equivalente con la Mean Reciprocal Rank (MRR)

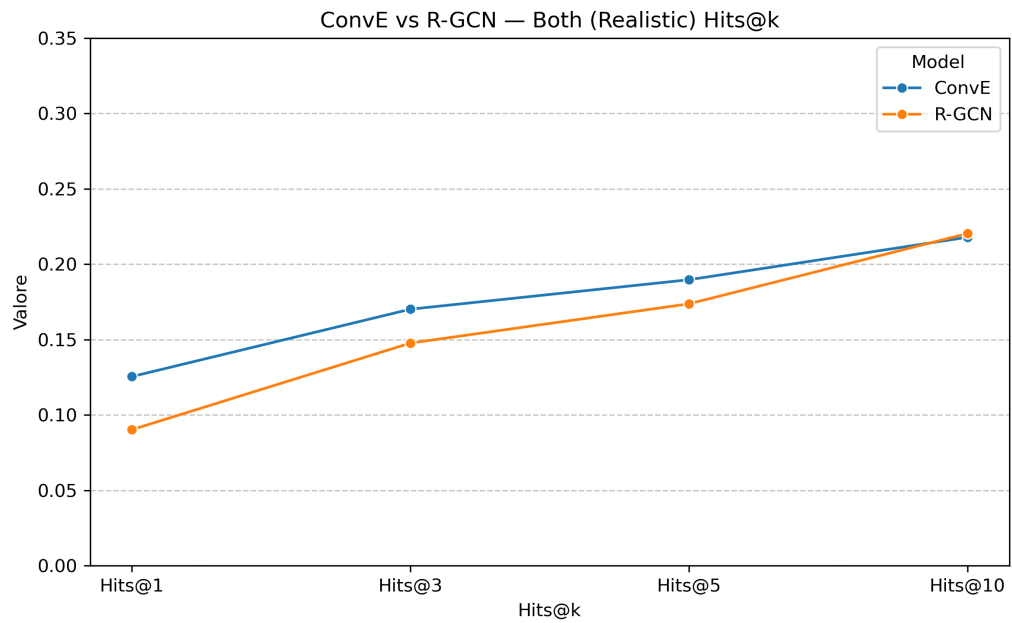


Figure 4.1: Linechart delle metriche Hits@K

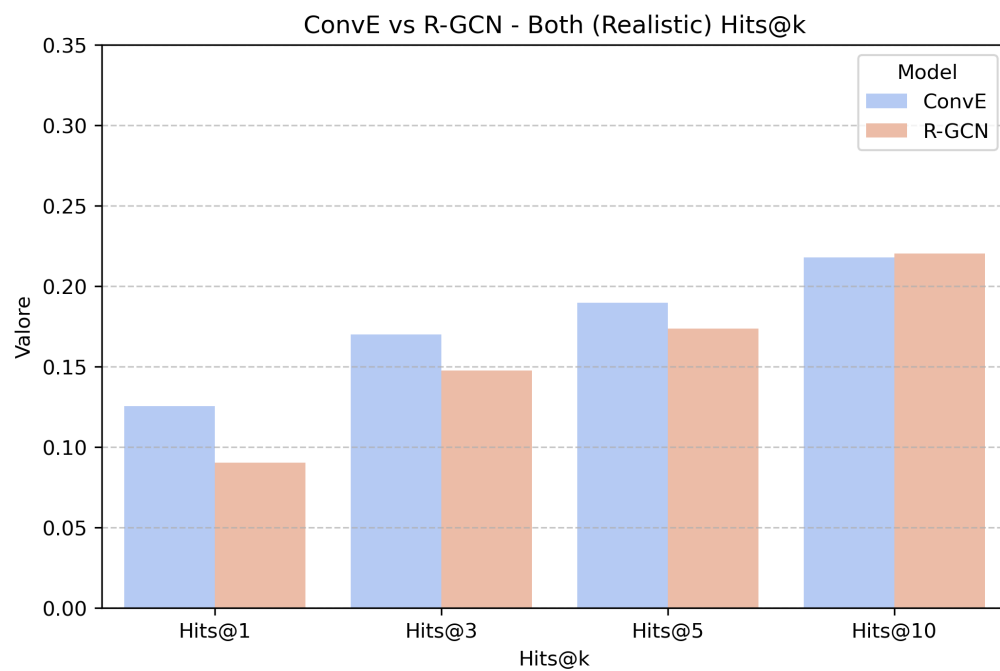


Figure 4.2: Barplot delle metriche Hits@K

5 — Discussione

5.1 Interpretazione dei risultati

Dall'analisi delle metriche Hits@K riportate in Tabella 4.1 si osserva che ConvE ottiene valori superiori rispetto a R-GCN su Hits@1, Hits@3 e Hits@5, IHR(MRR), con delta negativi rispetto a R-GCN, indicativo di una maggiore capacità del modello ConvE di recuperare correttamente i primi link predetti.

Tuttavia, per Hits@10, R-GCN supera leggermente ConvE, seppure con un delta molto contenuto (+0.002), suggerendo che R-GCN tende a distribuire le probabilità su un numero più ampio di candidate, migliorando leggermente le performance su K più grandi.

Le metriche di rank (Tabella 4.2), che misurano la posizione dei link corretti nella classifica, mostrano una chiara superiorità di R-GCN, con valori significativamente inferiori di Median Rank, AMR, GMR rispetto a ConvE. I delta negativi in verde indicano che R-GCN posiziona le entità corrette molto più in alto nella classifica, a dimostrazione di una maggiore precisione complessiva nella previsione dei link, soprattutto in scenari con numerose entità e relazioni complesse.

Se confrontiamo le metriche globali, emerge che la scelta del modello migliore dipende dall'obiettivo applicativo. ConvE è più efficace nel predire i link più probabili (Hits@1-5), mentre R-GCN garantisce una maggiore accuratezza complessiva (Median Rank, AMR, GMR).

Questo comportamento è coerente con la natura dei modelli perché ConvE è un approccio convoluzionale sui tensori relazionali, ottimizzato per catturare schemi locali e pattern ricorrenti, mentre R-GCN sfrutta la struttura del grafo per propagare informazioni lungo le relazioni, producendo embedding più robusti.

La superiorità di R-GCN nelle metriche di rank riflette anche l'impatto della

topologia del grafo sul modello. La capacità di aggregare informazioni dai vicini tramite message passing consente di catturare relazioni complesse e indirette che ConvE, operando su rappresentazioni convoluzionali, fatica a modellare. Questo effetto è evidente nei valori di AMR e GMR, che misurano rispettivamente il rank medio assoluto e globale delle previsioni corrette.

5.2 Considerazioni sulla stabilità dei modelli

Dalla Tabella 4.3 si nota un chiaro trade-off tra efficienza e accuratezza. ConvE presenta tempi di training estremamente bassi (31s) ma tempi di valutazione molto elevati (5810s), a causa della valutazione su tutti i possibili link inversi. Al contrario, R-GCN richiede un training più lungo (979s) ma tempi di evaluation molto bassi (51s), rendendo il modello più stabile e scalabile su grafi di grandi dimensioni. L'analisi dei delta (Δ) mostra che in termini di tempo totale, R-GCN è complessivamente più efficiente durante l'inferenza.

Riferimenti

- [1] Michael M. Bronstein et al. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. 2021. arXiv: 2104.13478 [cs.LG]. URL: <https://arxiv.org/abs/2104.13478>.
- [2] Zhe Chen et al. "Knowledge Graph Completion: A Review". In: *IEEE Access* 8 (2020), pp. 192435–192456. DOI: 10.1109/ACCESS.2020.3030076.
- [3] Yuanfei Dai et al. "A Survey on Knowledge Graph Embedding: Approaches, Applications and Benchmarks". In: *Electronics* 9.5 (2020). ISSN: 2079-9292. URL: <https://www.mdpi.com/2079-9292/9/5/750>.
- [4] Tim Dettmers et al. *Convolutional 2D Knowledge Graph Embeddings*. 2018. arXiv: 1707.01476 [cs.LG]. URL: <https://arxiv.org/abs/1707.01476>.
- [5] Ivan Diliso et al. *KG-SaF: Building Complete and Curated Datasets for Machine Learning and Reasoning on Knowledge Graphs*. Version 1.5.2. Dec. 2025. DOI: 10.5281/zenodo.17840023. URL: <https://doi.org/10.5281/zenodo.17840023>.
- [6] William L. Hamilton. *The Graph Neural Network Model*. 2020. URL: https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_mar29.pdf.
- [7] IBM. *Graph Neural Network*. URL: <https://www.ibm.com/think/topics/graph-neural-network>.
- [8] Andrea Rossi et al. "Knowledge Graph Embedding for Link Prediction: A Comparative Analysis". In: *ACM Transactions on Knowledge Discovery from Data* 15.2 (Jan. 2021), 1–49. ISSN: 1556-472X. DOI: 10.1145/3424672. URL: <http://dx.doi.org/10.1145/3424672>.

-
- [9] Andrea Rossi et al. "Knowledge Graph Embedding for Link Prediction: A Comparative Analysis". In: 15.2 (2021). ISSN: 1556-4681. DOI: [10.1145/3424672](https://doi.org/10.1145/3424672). URL: <https://doi.org/10.1145/3424672>.
 - [10] Benjamin Sanchez-Lengeling et al. *Introduction to GNN*. 2020. URL: <https://distill.pub/>.
 - [11] Michael Schlichtkrull et al. "Modeling relational data with graph convolutional networks". In: *European semantic web conference*. Springer. 2018, pp. 593–607.
 - [12] Thiviyan Thanapalasingam et al. "Relational graph convolutional networks: a closer look". In: *PeerJ Computer Science* 8 (2022), e1073.
 - [13] Jie Zhou et al. "Graph neural networks: A review of methods and applications". In: *AI Open* 1 (2020), pp. 57–81. ISSN: 2666-6510. DOI: <https://doi.org/10.1016/j.aiopen.2021.01.001>. URL: <https://www.sciencedirect.com/science/article/pii/S2666651021000012>.

